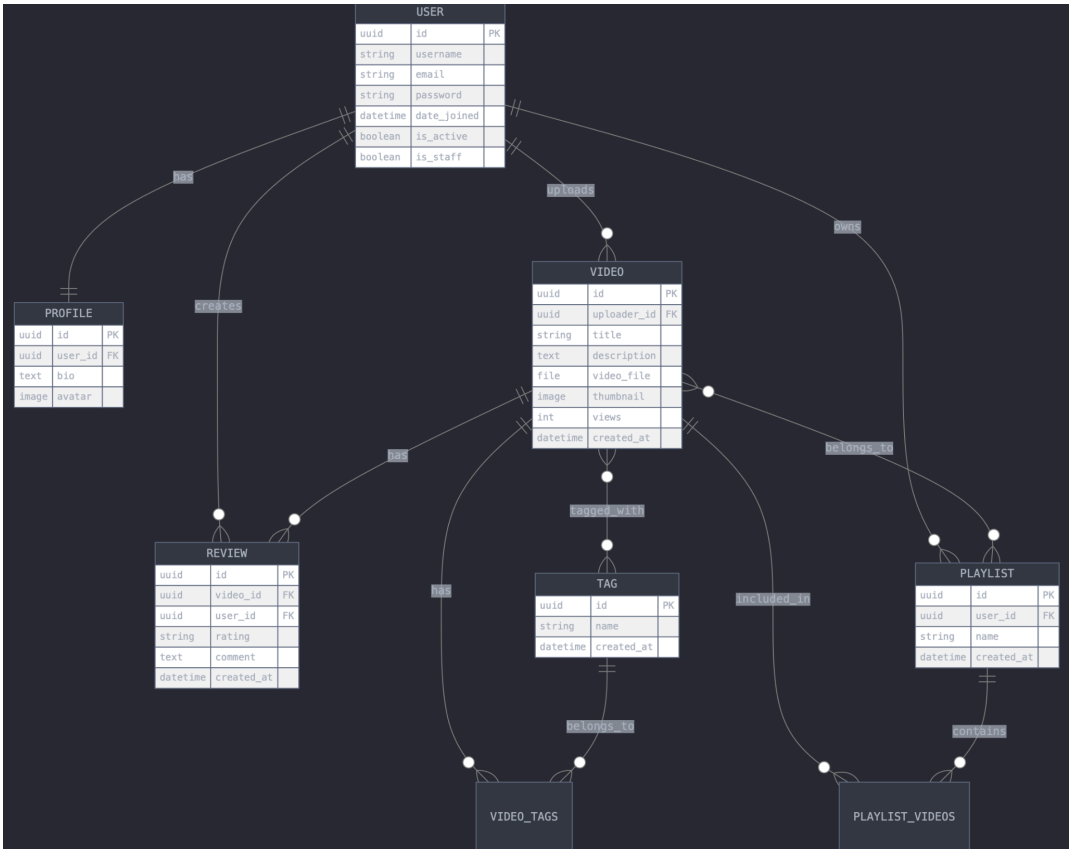
How The Apps Work Together:

1) Simplified to two main apps
2) More logical grouping of related features
3) Users app handles authentication and profiles
4) Projects app handles everything video-related

DB Schema



Key Changes and Relationships:

1) <u>User - Profile: One-to-One</u>
   Each User has exactly one Profile
   Each Profile belongs to exactly one User
   OneToOneField in Profile model

2) <u>User - Video: One-to-Many</u>
   One User can upload many Videos
   Each Video belongs to one User (uploader)
   Need to add ForeignKey to Video model

3) User - Review: One-to-Many
   One User can create many Reviews
   Each Review belongs to one User
   Need to add ForeignKey to Review model

4) User - Playlist: One-to-Many
   One User can create many Playlists
   Each Playlist belongs to one User
   Need to add ForeignKey to Playlist model

APPS

1."users" App (User Management)

```
users/
├── models.py        # User Profile model
├── views.py         # Login/Register logic
└── templates/       # User-related pages
```

Purpose-

1) Handles user accounts (signup/login)
2) Manages user profiles
3) Stores user information
4) Handles authentication

Key Models-

```
class Profile(models.Model):
    user = models.OneToOneField(User,on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
    avatar = models.ImageField(upload_to='avatars/')
    # Add more user-specific fields
```

2.”projects” App (Core Features)

```
projects/
  ├── models.py         # Video storage/metadata
  ├── views.py          # Video upload/playback logic
  └── templates/        # Video-related pages
```

Purpose-

1) Handles video uploads
2) Manages video playback
3) Stores video metadata
4) Manages video listings
5) Handles video reviews/ratings
6) Manages video tagging system

Key Models-

```
class Video(models.Model):
    id = models.UUIDField(
        default=uuid.uuid4,
        unique=True,
        primary_key=True,
        editable=False
    )
    title = models.CharField(
        max_length=100,
        help_text="Video title"
    )
    description = models.TextField(
        blank=True,
        help_text="Video description"
    )
    file = models.FileField(
        upload_to='videos/',
        validators=[FileExtensionValidator(['mp4', 'mov'])],
        help_text="Upload MP4 or MOV files only"
    )
    thumbnail = models.ImageField(
        upload_to='thumbnails/',
        blank=True,
        null=True,
        help_text="Video thumbnail"
    )
    views = models.PositiveIntegerField(default=0)
    tags = models.ManyToManyField('Tag', blank=True)
```

```
            created_at = models.DateTimeField(auto_now_add=True)

        def __str__(self):
            return self.title

class Review(models.Model):
        VOTING_CHOICES = (
            ('up', 'Upvote'),
            ('down', 'Downvote')
        )
        id = models.UUIDField(
            default=uuid.uuid4,
            unique=True,
            primary_key=True,
            editable=False
        )
        video = models.ForeignKey(Video, on_delete=models.CASCADE)
        rating = models.CharField(max_length=200, choices=VOTING_CHOICES)
        comment = models.TextField(
            blank=True,
            help_text="Review comment"
        )
        created_at = models.DateTimeField(auto_now_add=True)

        def __str__(self):
            return self.rating


class Tag(models.Model):
        id = models.UUIDField(
            default=uuid.uuid4,
            unique=True,
            primary_key=True,
            editable=False
        )
        name = models.CharField(max_length=200)
        created_at = models.DateTimeField(auto_now_add=True)

        def __str__(self):
            return self.name
```

```
class Playlist(models.Model):
        id = models.UUIDField(default=uuid.uuid4, primary_key=True,
                unique=True, editable=False)
        name = models.CharField(max_length=100)
        videos = models.ManyToManyField(Video)
        created_at = models.DateTimeField(auto_now_add=True)
        def __str__(self):
            return self.name
```